EDGE DEPLOYMENT TOOLS & OPERATIONS GUIDE

AI-Based Monitoring Solution for Oil & Gas Industry

Document Version: 1.0 **Date**: December 2025 **Audience**: Edge Engineers, Embedded Developers, IoT Engineers

---

TABLE OF CONTENTS

1. Edge Architecture Overview
2. Jetson Platform Setup
3. K3s Edge Kubernetes
4. TensorRT Edge Deployment
5. Edge Data Pipeline
6. MQTT & Edge Messaging
7. Edge Model Management
8. OTA Updates
9. Edge Monitoring
10. Complete Edge Workflows

---

1. EDGE ARCHITECTURE OVERVIEW

1.1 Edge Deployment Architecture

```
EDGE DEPLOYMENT ARCHITECTURE
============================

                  CLOUD
    +-----------------------------------------+
    |  +----------+  +----------+  +--------+  |
    |  | MLflow   |  | Harbor   |  | MQTT   |  |
    |  | Registry |  | Registry |  | Broker |  |
    |  +----------+  +----------+  +--------+  |
    +-----------------------------------------+
                    |
                    | (WAN/4G/5G)
                    |
    +-----------------------------------------+
    |               EDGE GATEWAY              |
    |  +--------+  +----------+  +----------+  |
    |  | K3s    |  | EdgeX    |  | InfluxDB |  |
    |  | Master |  | Foundry  |  | (Local)  |  |
    |  +--------+  +----------+  +----------+  |
    +-----------------------------------------+
                    |
        +-----------+-----------+
        |           |           |
    +-------+   +-------+   +-------+
    |Jetson |   |Jetson |   |Jetson |
    |AGX #1 |   |AGX #2 |   |Nano   |
```

```
    +-------+   +-------+   +-------+
        |           |           |
    +-------+   +-------+   +-------+
    |Sensors|   |Cameras|   |Sensors|
    +-------+   +-------+   +-------+
```

1.2 Edge Hardware Specifications

```
EDGE DEVICE SPECIFICATIONS
==========================

+------------------+-------------+-------------+-------------+
| Specification    | Jetson AGX  | Jetson Orin | Jetson Nano |
|                  | Orin 64GB   | NX 16GB     | 4GB         |
+------------------+-------------+-------------+-------------+
| GPU              | 2048 CUDA   | 1024 CUDA   | 128 CUDA    |
|                  | 64 Tensor   | 32 Tensor   | -           |
+------------------+-------------+-------------+-------------+
| AI Performance   | 275 TOPS    | 100 TOPS    | 0.5 TFLOPS  |
+------------------+-------------+-------------+-------------+
| CPU              | 12-core ARM | 8-core ARM  | 4-core ARM  |
|                  | Cortex-A78  | Cortex-A78  | Cortex-A57  |
+------------------+-------------+-------------+-------------+
| Memory           | 64GB        | 16GB        | 4GB         |
+------------------+-------------+-------------+-------------+
| Storage          | 64GB eMMC   | 16GB eMMC   | MicroSD     |
|                  | + NVMe      | + NVMe      |             |
+------------------+-------------+-------------+-------------+
| Power            | 15-60W      | 10-25W      | 5-10W       |
+------------------+-------------+-------------+-------------+
| Use Case         | Multi-model | Single      | Sensor      |
|                  | Inference   | Camera      | Gateway     |
+------------------+-------------+-------------+-------------+
```

---

2. JETSON PLATFORM SETUP

2.1 JetPack SDK Installation

```
# ============================================
# JETSON INITIAL SETUP
# ============================================

# Flash JetPack (from host Ubuntu machine)
# Download SDK Manager from NVIDIA
sdkmanager

# Or use command line
sdkmanager --cli --action install --login-type devzone \
    --product Jetson --target-os Linux \
    --version 5.1.2 --target JETSON_AGX_ORIN_TARGETS

# ============================================
```

```
# POST-FLASH CONFIGURATION
# ============================================

# Update system
sudo apt update && sudo apt upgrade -y

# Install essential packages
sudo apt install -y \
    python3-pip \
    python3-dev \
    libopenblas-base \
    libopenmpi-dev \
    libomp-dev \
    git \
    cmake \
    curl \
    wget

# Configure power mode
sudo nvpmodel -m 0  # MAXN mode (maximum performance)
sudo jetson_clocks   # Lock clocks at maximum

# Check Jetson status
sudo jtop  # Install with: sudo pip3 install jetson-stats

# ============================================
# CUDA & TENSORRT VERIFICATION
# ============================================

# Check CUDA
nvcc --version
ls /usr/local/cuda

# Check TensorRT
dpkg -l | grep tensorrt
python3 -c "import tensorrt; print(tensorrt.__version__)"

# Check cuDNN
cat /usr/include/cudnn_version.h | grep CUDNN_MAJOR -A 2

# ============================================
# PYTORCH FOR JETSON
# ============================================

# Install PyTorch for Jetson (pre-built wheel)
wget https://nvidia.box.com/shared/static/mp164asf3sceb570wvjsrezk1p4ftj8t.whl \
    -O torch-2.1.0-cp310-cp310-linux_aarch64.whl

pip3 install torch-2.1.0-cp310-cp310-linux_aarch64.whl

# Install torchvision
git clone --branch v0.16.0 https://github.com/pytorch/vision torchvision
cd torchvision
python3 setup.py install --user
```

2.2 Docker on Jetson

```
# ==========================================
# DOCKER RUNTIME SETUP
# ==========================================

# Add NVIDIA Container Runtime (already included in JetPack)
sudo apt install -y nvidia-container-runtime

# Configure Docker daemon
cat << EOF | sudo tee /etc/docker/daemon.json
{
    "runtimes": {
        "nvidia": {
            "path": "nvidia-container-runtime",
            "runtimeArgs": []
        }
    },
    "default-runtime": "nvidia"
}
EOF

sudo systemctl restart docker

# Test GPU access in container
docker run --rm --gpus all nvcr.io/nvidia/l4t-pytorch:r35.2.1-pth2.0-py3 \
    python3 -c "import torch; print(torch.cuda.is_available())"

# ==========================================
# BUILD ML INFERENCE CONTAINER
# ==========================================

# Dockerfile for Jetson
cat << 'EOF' > Dockerfile.jetson
FROM nvcr.io/nvidia/l4t-pytorch:r35.2.1-pth2.0-py3

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app

# Install dependencies
COPY requirements-edge.txt .
RUN pip3 install --no-cache-dir -r requirements-edge.txt

# Copy TensorRT models
COPY models/tensorrt/ ./models/

# Copy inference code
COPY src/ ./src/

# Health check
HEALTHCHECK --interval=30s --timeout=10s \
    CMD curl -f http://localhost:8080/health || exit 1
```

```
EXPOSE 8080

CMD ["python3", "src/edge_server.py"]
EOF

# Build
docker build -f Dockerfile.jetson -t oilgas/edge-inference:v1.0.0 .

# Run
docker run -d \
    --name edge-inference \
    --gpus all \
    --restart unless-stopped \
    -p 8080:8080 \
    -v /data/models:/app/models:ro \
    oilgas/edge-inference:v1.0.0
```

2.3 Jetson System Monitoring

```
#!/bin/bash
# scripts/jetson_monitor.sh
# Monitor Jetson device status

# Get device info
echo "=== Jetson Device Info ==="
cat /etc/nv_tegra_release

# Power mode
echo ""
echo "=== Power Mode ==="
sudo nvpmodel -q

# GPU status
echo ""
echo "=== GPU Status ==="
nvidia-smi 2>/dev/null || tegrastats --interval 1000 --stop

# CPU/GPU usage and temperature
echo ""
echo "=== Resource Usage ==="
tegrastats --interval 1000 &
TEGRA_PID=$!
sleep 5
kill $TEGRA_PID

# Disk usage
echo ""
echo "=== Disk Usage ==="
df -h /

# Memory
echo ""
echo "=== Memory ==="
```

```
free -h

# Docker containers
echo ""
echo "=== Docker Containers ==="
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

```
#!/usr/bin/env python3
# scripts/jetson_stats.py
"""
Jetson device statistics collector

Usage:
    python jetson_stats.py
    python jetson_stats.py --prometheus
"""

import subprocess
import json
import re
from typing import Dict
import argparse


def get_tegrastats() -> Dict:
    """Parse tegrastats output"""
    try:
        result = subprocess.run(
            ['tegrastats', '--interval', '1000'],
            capture_output=True,
            text=True,
            timeout=2
        )
        output = result.stdout.strip()
    except subprocess.TimeoutExpired:
        output = ""

    stats = {
        'cpu_usage': [],
        'gpu_usage': 0,
        'memory_used': 0,
        'memory_total': 0,
        'gpu_temp': 0,
        'cpu_temp': 0,
        'power_current': 0,
        'power_average': 0
    }

    # Parse RAM
    ram_match = re.search(r'RAM (\d+)/(\d+)MB', output)
    if ram_match:
        stats['memory_used'] = int(ram_match.group(1)) * 1024 * 1024
        stats['memory_total'] = int(ram_match.group(2)) * 1024 * 1024
```

```
    # Parse GPU
    gpu_match = re.search(r'GR3D_FREQ (\d+)%', output)
    if gpu_match:
        stats['gpu_usage'] = int(gpu_match.group(1))

    # Parse temperatures
    temp_match = re.search(r'GPU@(\d+\.?\d*)C', output)
    if temp_match:
        stats['gpu_temp'] = float(temp_match.group(1))

    cpu_temp_match = re.search(r'CPU@(\d+\.?\d*)C', output)
    if cpu_temp_match:
        stats['cpu_temp'] = float(cpu_temp_match.group(1))

    # Parse power
    power_match = re.search(r'VDD_GPU_SOC (\d+)mW/(\d+)mW', output)
    if power_match:
        stats['power_current'] = int(power_match.group(1))
        stats['power_average'] = int(power_match.group(2))

    return stats


def get_nvpmodel() -> str:
    """Get current power mode"""
    try:
        result = subprocess.run(
            ['sudo', 'nvpmodel', '-q'],
            capture_output=True,
            text=True
        )
        match = re.search(r'NV Power Mode: (\w+)', result.stdout)
        return match.group(1) if match else "unknown"
    except:
        return "unknown"


def format_prometheus(stats: Dict) -> str:
    """Format stats as Prometheus metrics"""
    lines = []

    lines.append(f'jetson_gpu_usage_percent {stats["gpu_usage"]}')
    lines.append(f'jetson_memory_used_bytes {stats["memory_used"]}')
    lines.append(f'jetson_memory_total_bytes {stats["memory_total"]}')
    lines.append(f'jetson_gpu_temperature_celsius {stats["gpu_temp"]}')
    lines.append(f'jetson_cpu_temperature_celsius {stats["cpu_temp"]}')
    lines.append(f'jetson_power_current_mw {stats["power_current"]}')
    lines.append(f'jetson_power_average_mw {stats["power_average"]}')

    return '\n'.join(lines)


def main():
    parser = argparse.ArgumentParser()
```

```
    parser.add_argument('--prometheus', action='store_true')
    parser.add_argument('--json', action='store_true')
    args = parser.parse_args()

    stats = get_tegrastats()
    stats['power_mode'] = get_nvpmodel()

    if args.prometheus:
        print(format_prometheus(stats))
    elif args.json:
        print(json.dumps(stats, indent=2))
    else:
        print(f"GPU Usage: {stats['gpu_usage']}%")
        print(f"Memory: {stats['memory_used'] / 1024 / 1024:.0f}MB / {stats['memory_total'] / 1024 / 10
        print(f"GPU Temp: {stats['gpu_temp']}C")
        print(f"CPU Temp: {stats['cpu_temp']}C")
        print(f"Power: {stats['power_current']}mW (avg: {stats['power_average']}mW)")
        print(f"Power Mode: {stats['power_mode']}")


if __name__ == '__main__':
    main()
```

---

3. K3S EDGE KUBERNETES

3.1 K3s Installation

```
# ============================================
# K3s MASTER NODE (Edge Gateway)
# ============================================

# Install K3s server
curl -sfL https://get.k3s.io | sh -s - \
    --write-kubeconfig-mode 644 \
    --disable traefik \
    --disable servicelb \
    --node-name edge-master \
    --tls-san edge-gateway.oilgas.internal

# Get node token for workers
sudo cat /var/lib/rancher/k3s/server/node-token

# Configure kubectl
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
kubectl get nodes

# ============================================
# K3s WORKER NODES (Jetson Devices)
# ============================================

# Install K3s agent on Jetson
K3S_URL="https://edge-gateway:6443"
```

```
K3S_TOKEN="node-token-from-master"

curl -sfL https://get.k3s.io | K3S_URL=$K3S_URL K3S_TOKEN=$K3S_TOKEN sh -s - \
    --node-name jetson-agx-001 \
    --node-label gpu=nvidia \
    --node-label device=jetson-agx-orin

# Verify
kubectl get nodes
kubectl describe node jetson-agx-001

# ==========================================
# NVIDIA GPU OPERATOR FOR K3s
# ==========================================

# Add NVIDIA Helm repo
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia

# Install GPU Operator
helm install gpu-operator nvidia/gpu-operator \
    --namespace gpu-operator \
    --create-namespace \
    --set operator.defaultRuntime=containerd \
    --set driver.enabled=false \
    --set toolkit.enabled=true

# Verify GPU availability
kubectl get pods -n gpu-operator
kubectl describe node jetson-agx-001 | grep nvidia
```

3.2 K3s Workload Deployment

```
# k3s/edge-inference-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edge-inference
  namespace: oilgas-edge
spec:
  replicas: 1
  selector:
    matchLabels:
      app: edge-inference
  template:
    metadata:
      labels:
        app: edge-inference
    spec:
      nodeSelector:
        device: jetson-agx-orin
      containers:
        - name: inference
          image: harbor.oilgas.internal/oilgas/edge-inference:v1.0.0
          ports:
```

```yaml
            - containerPort: 8080
          resources:
            limits:
              nvidia.com/gpu: 1
              memory: 8Gi
            requests:
              nvidia.com/gpu: 1
              memory: 4Gi
          volumeMounts:
            - name: models
              mountPath: /app/models
              readOnly: true
            - name: data
              mountPath: /data
          env:
            - name: DEVICE
              value: "cuda"
            - name: MODEL_PATH
              value: "/app/models/anomaly.engine"
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 60
            periodSeconds: 30
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 30
            periodSeconds: 10
      volumes:
        - name: models
          persistentVolumeClaim:
            claimName: edge-models-pvc
        - name: data
          hostPath:
            path: /data/inference
            type: DirectoryOrCreate
---
apiVersion: v1
kind: Service
metadata:
  name: edge-inference
  namespace: oilgas-edge
spec:
  type: NodePort
  selector:
    app: edge-inference
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30080
```

3.3 K3s Commands Reference

```
# ============================================
# K3s MANAGEMENT COMMANDS
# ============================================

# Check K3s status
sudo systemctl status k3s
sudo k3s kubectl get nodes

# View K3s logs
sudo journalctl -u k3s -f

# K3s crictl (container runtime)
sudo k3s crictl ps
sudo k3s crictl images
sudo k3s crictl logs <container-id>

# Restart K3s
sudo systemctl restart k3s

# Uninstall K3s
# Server: /usr/local/bin/k3s-uninstall.sh
# Agent: /usr/local/bin/k3s-agent-uninstall.sh

# ============================================
# KUBECTL COMMANDS
# ============================================

# Get resources
kubectl get pods -n oilgas-edge -o wide
kubectl get nodes --show-labels
kubectl top nodes
kubectl top pods -n oilgas-edge

# Logs
kubectl logs -f deployment/edge-inference -n oilgas-edge

# Exec into pod
kubectl exec -it deployment/edge-inference -n oilgas-edge -- /bin/bash

# Port forward
kubectl port-forward svc/edge-inference 8080:8080 -n oilgas-edge

# Apply manifests
kubectl apply -f k3s/
```

---

4. TENSORRT EDGE DEPLOYMENT

4.1 TensorRT Optimization for Edge

```
#!/usr/bin/env python3
# edge/optimize_tensorrt.py
```

```python
"""
TensorRT model optimization for edge deployment

Usage:
    python optimize_tensorrt.py --model models/anomaly.onnx --output models/anomaly.engine --fp16
"""

import argparse
import tensorrt as trt
import numpy as np
from pathlib import Path
from loguru import logger


class TensorRTOptimizer:
    """Optimize models for TensorRT on edge devices"""

    def __init__(self, verbose: bool = False):
        self.logger = trt.Logger(
            trt.Logger.VERBOSE if verbose else trt.Logger.WARNING
        )

    def optimize(
        self,
        onnx_path: str,
        output_path: str,
        fp16: bool = True,
        int8: bool = False,
        max_batch_size: int = 8,
        workspace_mb: int = 512,  # Reduced for edge
        dla_core: int = -1  # Deep Learning Accelerator (-1 = GPU only)
    ):
        """
        Optimize ONNX model for TensorRT.

        Args:
            onnx_path: Path to ONNX model
            output_path: Output TensorRT engine path
            fp16: Enable FP16 precision
            int8: Enable INT8 precision (requires calibration)
            max_batch_size: Maximum batch size
            workspace_mb: Workspace memory in MB
            dla_core: DLA core to use (-1 for GPU)
        """
        builder = trt.Builder(self.logger)
        config = builder.create_builder_config()

        # Set workspace
        config.set_memory_pool_limit(
            trt.MemoryPoolType.WORKSPACE,
            workspace_mb * (1 << 20)
        )

        # Enable FP16
```

```
        if fp16 and builder.platform_has_fast_fp16:
            config.set_flag(trt.BuilderFlag.FP16)
            logger.info("Enabled FP16 precision")

        # Enable INT8
        if int8 and builder.platform_has_fast_int8:
            config.set_flag(trt.BuilderFlag.INT8)
            logger.info("Enabled INT8 precision")

        # DLA (for Jetson)
        if dla_core >= 0:
            config.default_device_type = trt.DeviceType.DLA
            config.DLA_core = dla_core
            config.set_flag(trt.BuilderFlag.GPU_FALLBACK)
            logger.info(f"Using DLA core {dla_core}")

        # Create network
        network = builder.create_network(
            1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
        )

        # Parse ONNX
        parser = trt.OnnxParser(network, self.logger)

        with open(onnx_path, 'rb') as f:
            if not parser.parse(f.read()):
                for error in range(parser.num_errors):
                    logger.error(parser.get_error(error))
                raise RuntimeError("ONNX parsing failed")

        logger.info(f"Parsed ONNX model from {onnx_path}")
        logger.info(f"Network inputs: {network.num_inputs}")
        logger.info(f"Network outputs: {network.num_outputs}")

        # Build engine
        logger.info("Building TensorRT engine (this may take a while)...")
        serialized_engine = builder.build_serialized_network(network, config)

        if serialized_engine is None:
            raise RuntimeError("Engine build failed")

        # Save engine
        with open(output_path, 'wb') as f:
            f.write(serialized_engine)

        # Log engine info
        engine_size = Path(output_path).stat().st_size / (1024 * 1024)
        logger.info(f"Saved TensorRT engine to {output_path} ({engine_size:.1f} MB)")

        return output_path


class TensorRTInference:
    """TensorRT inference engine for edge deployment"""
```

```
def __init__(self, engine_path: str):
    import pycuda.driver as cuda
    import pycuda.autoinit

    self.cuda = cuda
    self.logger = trt.Logger(trt.Logger.WARNING)

    # Load engine
    with open(engine_path, 'rb') as f:
        runtime = trt.Runtime(self.logger)
        self.engine = runtime.deserialize_cuda_engine(f.read())

    self.context = self.engine.create_execution_context()

    # Setup I/O
    self._setup_io()

def _setup_io(self):
    """Setup input/output buffers"""
    self.inputs = []
    self.outputs = []
    self.bindings = []
    self.stream = self.cuda.Stream()

    for i in range(self.engine.num_io_tensors):
        name = self.engine.get_tensor_name(i)
        dtype = trt.nptype(self.engine.get_tensor_dtype(name))
        shape = self.engine.get_tensor_shape(name)

        # Handle dynamic shapes
        if -1 in shape:
            shape = tuple(max(1, s) for s in shape)

        size = trt.volume(shape)
        host_mem = self.cuda.pagelocked_empty(size, dtype)
        device_mem = self.cuda.mem_alloc(host_mem.nbytes)

        self.bindings.append(int(device_mem))

        if self.engine.get_tensor_mode(name) == trt.TensorIOMode.INPUT:
            self.inputs.append({
                'name': name,
                'host': host_mem,
                'device': device_mem,
                'shape': shape,
                'dtype': dtype
            })
        else:
            self.outputs.append({
                'name': name,
                'host': host_mem,
                'device': device_mem,
                'shape': shape,
```

```
                'dtype': dtype
            })

    def infer(self, input_data: np.ndarray) -> np.ndarray:
        """
        Run inference.

        Args:
            input_data: Input numpy array

        Returns:
            Output numpy array
        """
        # Copy input to host buffer
        np.copyto(self.inputs[0]['host'], input_data.ravel())

        # Transfer to device
        self.cuda.memcpy_htod_async(
            self.inputs[0]['device'],
            self.inputs[0]['host'],
            self.stream
        )

        # Set tensor addresses
        for inp in self.inputs:
            self.context.set_tensor_address(inp['name'], int(inp['device']))
        for out in self.outputs:
            self.context.set_tensor_address(out['name'], int(out['device']))

        # Execute
        self.context.execute_async_v3(stream_handle=self.stream.handle)

        # Transfer output to host
        self.cuda.memcpy_dtoh_async(
            self.outputs[0]['host'],
            self.outputs[0]['device'],
            self.stream
        )

        self.stream.synchronize()

        return self.outputs[0]['host'].reshape(self.outputs[0]['shape'])

    def benchmark(self, input_shape: tuple, iterations: int = 100) -> dict:
        """Benchmark inference latency"""
        import time

        dummy_input = np.random.randn(*input_shape).astype(np.float32)

        # Warmup
        for _ in range(10):
            self.infer(dummy_input)

        # Benchmark
```

```
        latencies = []
        for _ in range(iterations):
            start = time.perf_counter()
            self.infer(dummy_input)
            latencies.append((time.perf_counter() - start) * 1000)

        return {
            'mean_ms': np.mean(latencies),
            'std_ms': np.std(latencies),
            'p50_ms': np.percentile(latencies, 50),
            'p95_ms': np.percentile(latencies, 95),
            'p99_ms': np.percentile(latencies, 99),
            'throughput': 1000 / np.mean(latencies)
        }


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--model', required=True, help='ONNX model path')
    parser.add_argument('--output', required=True, help='Output engine path')
    parser.add_argument('--fp16', action='store_true')
    parser.add_argument('--int8', action='store_true')
    parser.add_argument('--dla', type=int, default=-1, help='DLA core')
    parser.add_argument('--workspace', type=int, default=512, help='Workspace MB')
    parser.add_argument('--benchmark', action='store_true')

    args = parser.parse_args()

    optimizer = TensorRTOptimizer(verbose=True)
    optimizer.optimize(
        args.model,
        args.output,
        fp16=args.fp16,
        int8=args.int8,
        dla_core=args.dla,
        workspace_mb=args.workspace
    )

    if args.benchmark:
        engine = TensorRTInference(args.output)
        results = engine.benchmark((1, 15), iterations=100)
        print("\nBenchmark Results:")
        print(f"  Mean: {results['mean_ms']:.2f}ms")
        print(f"  P95: {results['p95_ms']:.2f}ms")
        print(f"  Throughput: {results['throughput']:.0f} samples/sec")


if __name__ == '__main__':
    main()
```

---

5. EDGE DATA PIPELINE

5.1 Edge Data Collection

```python
#!/usr/bin/env python3
# edge/data_collector.py
"""
Edge sensor data collection

Usage:
    python data_collector.py --config edge_config.yaml
"""

import asyncio
import json
from datetime import datetime
from typing import Dict, List, Optional
import numpy as np
from loguru import logger


class EdgeDataCollector:
    """Collect data from edge sensors"""

    def __init__(self, config: Dict):
        self.config = config
        self.sensors = {}
        self.buffer = []
        self.buffer_size = config.get('buffer_size', 1000)

    async def connect_sensor(self, sensor_id: str, sensor_config: Dict):
        """Connect to a sensor"""
        sensor_type = sensor_config.get('type')

        if sensor_type == 'modbus':
            from pymodbus.client import ModbusTcpClient
            client = ModbusTcpClient(
                sensor_config['host'],
                port=sensor_config.get('port', 502)
            )
            client.connect()
            self.sensors[sensor_id] = {
                'client': client,
                'config': sensor_config
            }

        elif sensor_type == 'opcua':
            from asyncua import Client
            client = Client(sensor_config['url'])
            await client.connect()
            self.sensors[sensor_id] = {
                'client': client,
                'config': sensor_config
            }
```

```
        elif sensor_type == 'mqtt':
            import paho.mqtt.client as mqtt
            client = mqtt.Client()
            client.connect(
                sensor_config['broker'],
                sensor_config.get('port', 1883)
            )
            self.sensors[sensor_id] = {
                'client': client,
                'config': sensor_config
            }

        logger.info(f"Connected to sensor {sensor_id}")

    async def read_sensor(self, sensor_id: str) -> Dict:
        """Read data from a sensor"""
        sensor = self.sensors.get(sensor_id)
        if not sensor:
            return {}

        config = sensor['config']
        client = sensor['client']
        sensor_type = config.get('type')

        if sensor_type == 'modbus':
            # Read holding registers
            result = client.read_holding_registers(
                config['register'],
                config.get('count', 1),
                slave=config.get('slave_id', 1)
            )
            if result.isError():
                logger.error(f"Modbus read error: {result}")
                return {}

            return {
                'sensor_id': sensor_id,
                'timestamp': datetime.utcnow().isoformat(),
                'values': list(result.registers)
            }

        elif sensor_type == 'opcua':
            node = client.get_node(config['node_id'])
            value = await node.read_value()
            return {
                'sensor_id': sensor_id,
                'timestamp': datetime.utcnow().isoformat(),
                'value': value
            }

        return {}

    async def read_all_sensors(self) -> List[Dict]:
```

```
        """Read all sensors"""
        readings = []
        for sensor_id in self.sensors:
            try:
                reading = await self.read_sensor(sensor_id)
                if reading:
                    readings.append(reading)
            except Exception as e:
                logger.error(f"Error reading sensor {sensor_id}: {e}")
        return readings

    def add_to_buffer(self, data: Dict):
        """Add data to buffer"""
        self.buffer.append(data)
        if len(self.buffer) >= self.buffer_size:
            self.flush_buffer()

    def flush_buffer(self) -> List[Dict]:
        """Flush buffer and return data"""
        data = self.buffer.copy()
        self.buffer.clear()
        return data

    async def collect_loop(self, interval_ms: int = 100):
        """Main collection loop"""
        interval_sec = interval_ms / 1000

        while True:
            start_time = asyncio.get_event_loop().time()

            readings = await self.read_all_sensors()
            for reading in readings:
                self.add_to_buffer(reading)

            elapsed = asyncio.get_event_loop().time() - start_time
            sleep_time = max(0, interval_sec - elapsed)
            await asyncio.sleep(sleep_time)


class VibrationDataCollector:
    """High-frequency vibration data collector"""

    def __init__(self, device: str, sample_rate: int = 25600):
        self.device = device
        self.sample_rate = sample_rate
        self.window_size = sample_rate  # 1 second windows

    def read_waveform(self) -> np.ndarray:
        """Read vibration waveform"""
        # Simulated - replace with actual DAQ code
        # Example: NI DAQmx, PCB Piezotronics
        return np.random.randn(self.window_size).astype(np.float32)

    def compute_features(self, waveform: np.ndarray) -> Dict:
```

```
        """Compute vibration features from waveform"""
        from scipy import signal
        from scipy.stats import kurtosis, skew

        # Time domain features
        rms = np.sqrt(np.mean(waveform ** 2))
        peak = np.max(np.abs(waveform))
        crest_factor = peak / rms if rms > 0 else 0
        kurt = kurtosis(waveform)
        skewness = skew(waveform)

        # Frequency domain
        freqs, psd = signal.welch(waveform, self.sample_rate, nperseg=1024)

        # Find dominant frequencies
        peak_idx = np.argsort(psd)[-5:]
        peak_freqs = freqs[peak_idx]
        peak_powers = psd[peak_idx]

        return {
            'rms': float(rms),
            'peak': float(peak),
            'crest_factor': float(crest_factor),
            'kurtosis': float(kurt),
            'skewness': float(skewness),
            'dominant_freq_1': float(peak_freqs[-1]),
            'dominant_freq_2': float(peak_freqs[-2]),
            'dominant_freq_3': float(peak_freqs[-3]),
            'power_1': float(peak_powers[-1]),
            'power_2': float(peak_powers[-2]),
            'power_3': float(peak_powers[-3])
        }
```

---

6. MQTT & EDGE MESSAGING

6.1 MQTT Client

```
#!/usr/bin/env python3
# edge/mqtt_client.py
"""
MQTT client for edge messaging

Usage:
    python mqtt_client.py --broker mqtt.oilgas.internal --topic edge/sensors/#
"""

import json
import os
from datetime import datetime
from typing import Callable, Dict, Optional
import paho.mqtt.client as mqtt
from loguru import logger
```

```python
class EdgeMQTTClient:
    """MQTT client for edge communication"""

    def __init__(
        self,
        broker: str,
        port: int = 1883,
        client_id: str = None,
        username: str = None,
        password: str = None,
        use_tls: bool = False
    ):
        self.broker = broker
        self.port = port

        self.client = mqtt.Client(
            client_id=client_id or f"edge-{os.uname().nodename}",
            protocol=mqtt.MQTTv5
        )

        if username and password:
            self.client.username_pw_set(username, password)

        if use_tls:
            self.client.tls_set()

        # Callbacks
        self.client.on_connect = self._on_connect
        self.client.on_disconnect = self._on_disconnect
        self.client.on_message = self._on_message

        self.message_handlers: Dict[str, Callable] = {}
        self.connected = False

    def _on_connect(self, client, userdata, flags, rc, properties=None):
        """Connection callback"""
        if rc == 0:
            logger.info(f"Connected to MQTT broker {self.broker}")
            self.connected = True

            # Resubscribe to topics
            for topic in self.message_handlers:
                self.client.subscribe(topic)
        else:
            logger.error(f"Connection failed with code {rc}")

    def _on_disconnect(self, client, userdata, rc, properties=None):
        """Disconnection callback"""
        logger.warning(f"Disconnected from MQTT broker (rc={rc})")
        self.connected = False

    def _on_message(self, client, userdata, msg):
```

```
        """Message callback"""
        topic = msg.topic
        try:
            payload = json.loads(msg.payload.decode())
        except json.JSONDecodeError:
            payload = msg.payload.decode()

        # Find matching handler
        for pattern, handler in self.message_handlers.items():
            if mqtt.topic_matches_sub(pattern, topic):
                try:
                    handler(topic, payload)
                except Exception as e:
                    logger.error(f"Handler error for {topic}: {e}")

    def connect(self):
        """Connect to broker"""
        self.client.connect(self.broker, self.port, keepalive=60)
        self.client.loop_start()

    def disconnect(self):
        """Disconnect from broker"""
        self.client.loop_stop()
        self.client.disconnect()

    def subscribe(self, topic: str, handler: Callable):
        """Subscribe to topic with handler"""
        self.message_handlers[topic] = handler
        if self.connected:
            self.client.subscribe(topic)
        logger.info(f"Subscribed to {topic}")

    def publish(self, topic: str, payload: Dict, qos: int = 1, retain: bool = False):
        """Publish message"""
        message = json.dumps(payload)
        result = self.client.publish(topic, message, qos=qos, retain=retain)
        return result.is_published()

    def publish_sensor_data(self, asset_id: str, sensor_type: str, data: Dict):
        """Publish sensor data"""
        topic = f"edge/sensors/{asset_id}/{sensor_type}"
        payload = {
            'timestamp': datetime.utcnow().isoformat(),
            'asset_id': asset_id,
            'sensor_type': sensor_type,
            **data
        }
        return self.publish(topic, payload)

    def publish_prediction(self, asset_id: str, model: str, prediction: Dict):
        """Publish ML prediction"""
        topic = f"edge/predictions/{asset_id}/{model}"
        payload = {
            'timestamp': datetime.utcnow().isoformat(),
```

```
            'asset_id': asset_id,
            'model': model,
            **prediction
        }
        return self.publish(topic, payload)

    def publish_alert(self, asset_id: str, severity: str, message: str, details: Dict = None):
        """Publish alert"""
        topic = f"edge/alerts/{severity}/{asset_id}"
        payload = {
            'timestamp': datetime.utcnow().isoformat(),
            'asset_id': asset_id,
            'severity': severity,
            'message': message,
            'details': details or {}
        }
        return self.publish(topic, payload, qos=2)  # Exactly once delivery


# Example usage
def main():
    client = EdgeMQTTClient(
        broker="mqtt.oilgas.internal",
        username="edge-device",
        password="password"
    )

    # Message handler
    def on_command(topic, payload):
        logger.info(f"Received command: {topic} -> {payload}")

    # Connect and subscribe
    client.connect()
    client.subscribe("edge/commands/#", on_command)

    # Publish sensor data
    import time
    import random

    while True:
        client.publish_sensor_data(
            asset_id="PUMP-001",
            sensor_type="vibration",
            data={
                'rms': random.uniform(0, 10),
                'peak': random.uniform(0, 20),
                'temperature': random.uniform(40, 80)
            }
        )
        time.sleep(1)


if __name__ == '__main__':
    main()
```

7. EDGE MODEL MANAGEMENT

7.1 Model Sync Service

```
#!/usr/bin/env python3
# edge/model_sync.py
"""
Edge model synchronization service

Usage:
    python model_sync.py --config model_sync.yaml
"""

import os
import hashlib
import requests
from pathlib import Path
from typing import Dict, Optional
import yaml
from loguru import logger


class EdgeModelManager:
    """Manage ML models on edge devices"""

    def __init__(self, config: Dict):
        self.config = config
        self.model_dir = Path(config.get('model_dir', '/models'))
        self.model_registry_url = config.get('registry_url')
        self.current_models: Dict[str, Dict] = {}

        self.model_dir.mkdir(parents=True, exist_ok=True)
        self._load_manifest()

    def _load_manifest(self):
        """Load local model manifest"""
        manifest_path = self.model_dir / 'manifest.yaml'
        if manifest_path.exists():
            with open(manifest_path) as f:
                self.current_models = yaml.safe_load(f) or {}

    def _save_manifest(self):
        """Save local model manifest"""
        manifest_path = self.model_dir / 'manifest.yaml'
        with open(manifest_path, 'w') as f:
            yaml.dump(self.current_models, f)

    def _compute_checksum(self, file_path: Path) -> str:
        """Compute file checksum"""
        sha256 = hashlib.sha256()
        with open(file_path, 'rb') as f:
            for chunk in iter(lambda: f.read(8192), b''):
```

```
            sha256.update(chunk)
    return sha256.hexdigest()

def check_for_updates(self) -> Dict[str, Dict]:
    """Check registry for model updates"""
    try:
        response = requests.get(
            f"{self.model_registry_url}/api/models",
            timeout=30
        )
        response.raise_for_status()
        remote_models = response.json()

        updates = {}
        for model_name, model_info in remote_models.items():
            local_info = self.current_models.get(model_name, {})

            if model_info.get('version') != local_info.get('version'):
                updates[model_name] = model_info

        return updates

    except Exception as e:
        logger.error(f"Failed to check for updates: {e}")
        return {}

def download_model(self, model_name: str, model_info: Dict) -> bool:
    """Download model from registry"""
    try:
        download_url = model_info.get('download_url')
        version = model_info.get('version')

        logger.info(f"Downloading {model_name} v{version}")

        response = requests.get(download_url, stream=True, timeout=300)
        response.raise_for_status()

        # Save to temp file first
        temp_path = self.model_dir / f"{model_name}.tmp"
        with open(temp_path, 'wb') as f:
            for chunk in response.iter_content(chunk_size=8192):
                f.write(chunk)

        # Verify checksum
        checksum = self._compute_checksum(temp_path)
        if checksum != model_info.get('checksum'):
            logger.error(f"Checksum mismatch for {model_name}")
            temp_path.unlink()
            return False

        # Move to final location
        final_path = self.model_dir / model_info.get('filename', f"{model_name}.engine")
        temp_path.rename(final_path)
```

```
            # Update manifest
            self.current_models[model_name] = {
                'version': version,
                'checksum': checksum,
                'path': str(final_path),
                'downloaded_at': datetime.utcnow().isoformat()
            }
            self._save_manifest()

            logger.info(f"Downloaded {model_name} v{version}")
            return True

        except Exception as e:
            logger.error(f"Failed to download {model_name}: {e}")
            return False

    def sync_models(self) -> Dict[str, bool]:
        """Sync all models with registry"""
        updates = self.check_for_updates()
        results = {}

        for model_name, model_info in updates.items():
            results[model_name] = self.download_model(model_name, model_info)

        return results

    def get_model_path(self, model_name: str) -> Optional[Path]:
        """Get path to model file"""
        model_info = self.current_models.get(model_name)
        if model_info:
            return Path(model_info['path'])
        return None

    def list_models(self) -> Dict:
        """List available models"""
        return self.current_models
```

---

8. OTA UPDATES

8.1 OTA Update Service

```
#!/usr/bin/env python3
# edge/ota_update.py
"""
Over-The-Air update service for edge devices

Usage:
    python ota_update.py check
    python ota_update.py apply
"""

import os
```

```python
import subprocess
import hashlib
import requests
from pathlib import Path
from typing import Dict, Optional
from loguru import logger


class OTAUpdateService:
    """OTA update service for edge devices"""

    def __init__(self, config: Dict):
        self.config = config
        self.update_server = config.get('update_server')
        self.device_id = config.get('device_id', os.uname().nodename)
        self.current_version = self._get_current_version()

    def _get_current_version(self) -> str:
        """Get current software version"""
        version_file = Path('/etc/edge-version')
        if version_file.exists():
            return version_file.read_text().strip()
        return "0.0.0"

    def check_update(self) -> Optional[Dict]:
        """Check for available updates"""
        try:
            response = requests.post(
                f"{self.update_server}/api/check-update",
                json={
                    'device_id': self.device_id,
                    'current_version': self.current_version,
                    'device_type': self._get_device_type()
                },
                timeout=30
            )
            response.raise_for_status()

            update_info = response.json()
            if update_info.get('update_available'):
                return update_info

            return None

        except Exception as e:
            logger.error(f"Update check failed: {e}")
            return None

    def _get_device_type(self) -> str:
        """Get device type"""
        # Check for Jetson
        if Path('/etc/nv_tegra_release').exists():
            with open('/etc/nv_tegra_release') as f:
                content = f.read()
```

```
                if 'AGX' in content:
                    return 'jetson-agx-orin'
                elif 'NX' in content:
                    return 'jetson-orin-nx'
                elif 'Nano' in content:
                    return 'jetson-nano'
        return 'generic-linux'

    def download_update(self, update_info: Dict) -> Optional[Path]:
        """Download update package"""
        try:
            download_url = update_info.get('download_url')
            expected_checksum = update_info.get('checksum')

            logger.info(f"Downloading update v{update_info.get('version')}")

            response = requests.get(download_url, stream=True, timeout=600)
            response.raise_for_status()

            update_path = Path(f"/tmp/update-{update_info.get('version')}.tar.gz")

            with open(update_path, 'wb') as f:
                for chunk in response.iter_content(chunk_size=8192):
                    f.write(chunk)

            # Verify checksum
            sha256 = hashlib.sha256()
            with open(update_path, 'rb') as f:
                for chunk in iter(lambda: f.read(8192), b''):
                    sha256.update(chunk)

            if sha256.hexdigest() != expected_checksum:
                logger.error("Update checksum verification failed")
                update_path.unlink()
                return None

            logger.info("Update downloaded and verified")
            return update_path

        except Exception as e:
            logger.error(f"Download failed: {e}")
            return None

    def apply_update(self, update_path: Path, update_info: Dict) -> bool:
        """Apply update"""
        try:
            version = update_info.get('version')
            logger.info(f"Applying update v{version}")

            # Extract update
            subprocess.run(
                ['tar', 'xzf', str(update_path), '-C', '/tmp/update'],
                check=True
            )
```

```
            # Run pre-update script
            pre_script = Path('/tmp/update/pre-update.sh')
            if pre_script.exists():
                subprocess.run(['bash', str(pre_script)], check=True)

            # Run main update script
            update_script = Path('/tmp/update/update.sh')
            if update_script.exists():
                subprocess.run(['bash', str(update_script)], check=True)

            # Update version file
            Path('/etc/edge-version').write_text(version)

            # Run post-update script
            post_script = Path('/tmp/update/post-update.sh')
            if post_script.exists():
                subprocess.run(['bash', str(post_script)], check=True)

            # Report success
            self._report_update_status(version, success=True)

            logger.info(f"Update v{version} applied successfully")
            return True

        except Exception as e:
            logger.error(f"Update failed: {e}")
            self._report_update_status(update_info.get('version'), success=False, error=str(e))
            return False

        finally:
            # Cleanup
            if update_path.exists():
                update_path.unlink()

    def _report_update_status(self, version: str, success: bool, error: str = None):
        """Report update status to server"""
        try:
            requests.post(
                f"{self.update_server}/api/update-status",
                json={
                    'device_id': self.device_id,
                    'version': version,
                    'success': success,
                    'error': error
                },
                timeout=30
            )
        except:
            pass
```

---

9. EDGE MONITORING

9.1 Edge Metrics Exporter

```
#!/usr/bin/env python3
# edge/metrics_exporter.py
"""
Prometheus metrics exporter for edge devices

Usage:
    python metrics_exporter.py --port 9100
"""

from prometheus_client import start_http_server, Gauge, Counter, Histogram, Info
import time
import subprocess
import re
from loguru import logger


class EdgeMetricsExporter:
    """Export edge device metrics to Prometheus"""

    def __init__(self, port: int = 9100):
        self.port = port

        # System metrics
        self.cpu_usage = Gauge('edge_cpu_usage_percent', 'CPU usage percentage', ['core'])
        self.memory_used = Gauge('edge_memory_used_bytes', 'Memory used bytes')
        self.memory_total = Gauge('edge_memory_total_bytes', 'Total memory bytes')
        self.disk_used = Gauge('edge_disk_used_bytes', 'Disk used bytes', ['mountpoint'])
        self.disk_total = Gauge('edge_disk_total_bytes', 'Total disk bytes', ['mountpoint'])

        # GPU metrics (Jetson)
        self.gpu_usage = Gauge('edge_gpu_usage_percent', 'GPU usage percentage')
        self.gpu_memory_used = Gauge('edge_gpu_memory_used_bytes', 'GPU memory used')
        self.gpu_memory_total = Gauge('edge_gpu_memory_total_bytes', 'GPU total memory')
        self.gpu_temperature = Gauge('edge_gpu_temperature_celsius', 'GPU temperature')
        self.cpu_temperature = Gauge('edge_cpu_temperature_celsius', 'CPU temperature')

        # Power metrics
        self.power_current = Gauge('edge_power_current_mw', 'Current power draw mW')
        self.power_average = Gauge('edge_power_average_mw', 'Average power draw mW')

        # Inference metrics
        self.inference_count = Counter('edge_inference_total', 'Total inferences', ['model'])
        self.inference_latency = Histogram(
            'edge_inference_duration_seconds',
            'Inference duration',
            ['model'],
            buckets=[0.001, 0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1.0]
        )

        # Device info
```

```
        self.device_info = Info('edge_device', 'Edge device information')

    def collect_jetson_stats(self):
        """Collect Jetson-specific metrics"""
        try:
            result = subprocess.run(
                ['tegrastats', '--interval', '1000'],
                capture_output=True,
                text=True,
                timeout=2
            )
            output = result.stdout.strip()

            # Parse RAM
            ram_match = re.search(r'RAM (\d+)/(\d+)MB', output)
            if ram_match:
                self.memory_used.set(int(ram_match.group(1)) * 1024 * 1024)
                self.memory_total.set(int(ram_match.group(2)) * 1024 * 1024)

            # Parse GPU
            gpu_match = re.search(r'GR3D_FREQ (\d+)%', output)
            if gpu_match:
                self.gpu_usage.set(int(gpu_match.group(1)))

            # Parse temperatures
            gpu_temp_match = re.search(r'GPU@(\d+\.?\d*)C', output)
            if gpu_temp_match:
                self.gpu_temperature.set(float(gpu_temp_match.group(1)))

            cpu_temp_match = re.search(r'CPU@(\d+\.?\d*)C', output)
            if cpu_temp_match:
                self.cpu_temperature.set(float(cpu_temp_match.group(1)))

            # Parse power
            power_match = re.search(r'VDD_GPU_SOC (\d+)mW/(\d+)mW', output)
            if power_match:
                self.power_current.set(int(power_match.group(1)))
                self.power_average.set(int(power_match.group(2)))

        except subprocess.TimeoutExpired:
            pass
        except Exception as e:
            logger.error(f"Failed to collect Jetson stats: {e}")

    def collect_system_stats(self):
        """Collect general system metrics"""
        import psutil

        # CPU
        for i, percent in enumerate(psutil.cpu_percent(percpu=True)):
            self.cpu_usage.labels(core=str(i)).set(percent)

        # Memory
        mem = psutil.virtual_memory()
```

```
            self.memory_used.set(mem.used)
            self.memory_total.set(mem.total)

            # Disk
            for partition in psutil.disk_partitions():
                try:
                    usage = psutil.disk_usage(partition.mountpoint)
                    self.disk_used.labels(mountpoint=partition.mountpoint).set(usage.used)
                    self.disk_total.labels(mountpoint=partition.mountpoint).set(usage.total)
                except:
                    pass

        def set_device_info(self):
            """Set device information"""
            import platform

            info = {
                'hostname': platform.node(),
                'platform': platform.system(),
                'architecture': platform.machine(),
                'python_version': platform.python_version()
            }

            # Add Jetson info
            if Path('/etc/nv_tegra_release').exists():
                with open('/etc/nv_tegra_release') as f:
                    info['jetpack'] = f.read().strip()

            self.device_info.info(info)

        def run(self):
            """Start metrics server"""
            start_http_server(self.port)
            self.set_device_info()
            logger.info(f"Metrics server started on port {self.port}")

            while True:
                self.collect_system_stats()
                self.collect_jetson_stats()
                time.sleep(15)


    if __name__ == '__main__':
        import argparse
        parser = argparse.ArgumentParser()
        parser.add_argument('--port', type=int, default=9100)
        args = parser.parse_args()

        exporter = EdgeMetricsExporter(args.port)
        exporter.run()
```

---

10. COMPLETE EDGE WORKFLOWS

10.1 Edge Deployment Script

```
#!/bin/bash
# workflows/deploy_edge.sh
# Deploy ML inference to edge device

set -e

DEVICE_IP="${1:?Usage: $0 <device_ip> [version]}"
VERSION="${2:-latest}"

echo "Deploying to edge device $DEVICE_IP"
echo "========================================"

# 1. Check connectivity
echo "Step 1: Checking connectivity..."
ssh edge@$DEVICE_IP "echo Connected"

# 2. Sync models
echo "Step 2: Syncing models..."
rsync -avz models/tensorrt/ edge@$DEVICE_IP:/data/models/

# 3. Deploy container
echo "Step 3: Deploying container..."
ssh edge@$DEVICE_IP << EOF
docker pull harbor.oilgas.internal/oilgas/edge-inference:$VERSION
docker stop edge-inference || true
docker rm edge-inference || true
docker run -d \
    --name edge-inference \
    --gpus all \
    --restart unless-stopped \
    -p 8080:8080 \
    -v /data/models:/app/models:ro \
    harbor.oilgas.internal/oilgas/edge-inference:$VERSION
EOF

# 4. Verify deployment
echo "Step 4: Verifying deployment..."
sleep 10
curl -f http://$DEVICE_IP:8080/health

echo ""
echo "Deployment complete!"
```

10.2 Quick Reference

```
# ============================================
# EDGE QUICK REFERENCE
# ============================================

# --- JETSON ---
sudo nvpmodel -q                    # Check power mode
```

```
sudo nvpmodel -m 0                    # Set MAXN mode
sudo jetson_clocks                    # Lock clocks
tegrastats                            # Real-time stats
jtop                                 # Interactive monitor

# --- K3s ---
sudo k3s kubectl get nodes
sudo k3s kubectl get pods -A
sudo k3s crictl ps

# --- TENSORRT ---
trtexec --onnx=model.onnx --saveEngine=model.engine --fp16
trtexec --loadEngine=model.engine --batch=1

# --- DOCKER ---
docker run --gpus all --rm nvidia/cuda:12.1-base nvidia-smi
docker logs -f edge-inference

# --- MQTT ---
mosquitto_pub -h mqtt -t "edge/test" -m "hello"
mosquitto_sub -h mqtt -t "edge/#" -v

# --- SYNC ---
rsync -avz models/ edge@device:/models/
ssh edge@device "docker restart edge-inference"
```

END OF EDGE DEPLOYMENT TOOLS & OPERATIONS GUIDE